

newtons (generic function with 3 methods)

```
1 function newtons(f:: Function, derivative:: Function, x0 :: Float64, tol::Float64
  = 0.5 * 10^-8, maxIter = nothing)
2     xi = x0
3     value = f(xi)
4     iter = 0
5
6     # If max iteration set otherwise simply continue till tolerance is satisfied
7     while (maxIter != nothing && iter < maxIter) || (maxIter == nothing)
8         xLast = xi
9         xi = xLast - value/derivative(xLast)
10        value = f(xi)
11
12        if abs(xi - xLast) < tol || value == 0.0
13            break
14        end
15        iter+=1
16    end
17
18    return (xi, value)
19 end
```

► (-0.970899, 0.0)

```
1 begin
2     f1(x) = sin(x) - 6x - 5
3     df1(x) = cos(x) - 6
4
5     ans1, val1 = newtons(f1, df1, 0.0)
6 end
```

Section 1.4: Computer Problem 2b:

Use Newton's method to approximate the root to eight correct decimal places of $\sin(x) = 6x + 5$.

Answer

$$\sin(x) = 6x + 5 \rightarrow f(x) = \sin(x) - 6x - 5$$

r is the root of $f(x)$. $r = -0.9708989235042558$.

modifiedNewtons (generic function with 3 methods)

```
1 function modifiedNewtons(f:: Function, derivative:: Function, x0 :: Float64,  
multiplicity :: Float64 = 1.0, tol::Float64 = 0.5 * 10^-8)  
2     xi = x0  
3     value = f(xi)  
4     iter = 0  
5     while true  
6         xLast = xi  
7         xi = xLast - value/derivative(xLast) * multiplicity  
8         value = f(xi)  
9  
10        if abs(xi - xLast) < tol || value == 0.0  
11            break  
12        end  
13        iter+=1  
14    end  
15  
16    return (xi, value)  
17 end
```

Section 1.4: Computer Problem 4a:

Apply Newton's Method to find the only root to as much accuracy as possible, and find the root's multiplicity. Then use Modified Newton's Method to converge to the root quadratically. Report the forward and backward errors of the best approximation obtained from each method:

$$f(x) = 2e^{x-1} - x^2 - 1$$

Answer

Root found with normal Newton's Method $r_n = 0.9999932833601749$

$$f(x) = 2e^{x-1} - x^2 - 1$$

$$f'(x) = 2e^{x-1} - 2x \implies f'(r_n) = 4.511324647182846e - 11 \approx 0$$

$$f''(x) = 2e^{x-1} - 2 \implies f''(r_n) = -1.3433234536952554e - 5 \approx 0$$

$$f'''(x) = 2e^{x-1} \implies f'''(r_n) = 1.999986566765463 \approx 2$$

Multiplicity $m = 3$

Comparing to results with Theorem 1.12.

$$S = 0.6660922953192595 \approx \frac{2}{3} = \frac{m-1}{m}. \text{ Thus } m = 3.$$

Root found with modified newton's Method $r_m = 1.0000000207213324$

$$\text{Forward error with normal newton's Method: } |1 - r_n| = 6.716639825099513e - 6$$

$$\text{Backward error with normal Newton's Method} = |f(r_n)| = 0.0$$

$$\text{Forward error with modified newton's Method: } |1 - r_m| = 2.0721332427697803e - 8$$

$$\text{Backward error with normal Newton's Method} = |f(r_n)| = 0.0$$

```
► (0.999993, 0.0, 1.0, 0.0)
```

```
1 begin
2     f2(x) = 2exp(x - 1) - x^2 - 1
3     df2(x) = 2exp(x - 1) - 2x
4     d2f2(x) = 2exp(x - 1) - 2
5     d3f2(x) = 2exp(x - 1)
6
7     root2, val2 = newtons(f2, df2, 0.0, 0.5 * 10^-8)
8
9
10    e10 = abs(newtons(f2, df2, 0.0, 0.5 * 10^-8, 10)[1] - root2)
11    e11 = abs(newtons(f2, df2, 0.0, 0.5 * 10^-8, 11)[1] - root2)
12
13    S = e11/e10
14
15    modifiedRoot2, modifiedVal2 = modifiedNewtons(f2, df2, 0.0, 3.0, 0.5 * 10^-8)
16
17    (root2, val2, modifiedRoot2, modifiedVal2)
18 end
```

Section 1.4: Computer Problem 6:

A 10-cm-high cone contains 60cm^3 of ice cream, including a hemispherical scoop on top. Find the radius of the scoop to four correct decimal places.

Answer

$$h = 10$$

$$V(r) = V_{\text{cone}}(r) + V_{\text{hemisphere}}(r) = \frac{1}{3}\pi r^2 h + \frac{1}{2} \frac{4}{3}\pi r^3 = \frac{1}{3}\pi r^2 h + \frac{2}{3}\pi r^3 = \frac{10}{3}\pi r^2 + \frac{2}{3}\pi r^3$$

$$V(r) = 60$$

$$g(r) = V(r) - 60 = \frac{10}{3}\pi r^2 + \frac{2}{3}\pi r^3 - 60$$

$$x_0 = 2$$

$$g'(r) = \frac{20}{3}\pi r + 2\pi r^2$$

The radius to 4 correct decimal places is 2.0201073212197014 cm.

```
►Dict("radius" => 2.02011, "value" => 1.42109e-14)
```

```
1 begin
2   g(r :: Float64) = pi * 10/3 * r^2 + 2/3 * pi * r^3 - 60
3   dg(r :: Float64) = pi * 20/3 * r + 2 * pi * r^2
4
5   radius, value = newtons(g, dg, 2.0, 0.5*10^-4)
6
7   Dict("radius" => radius, "value" => value)
8 end
```